Genetic Algorithm Optimization Analysis

This notebook analyses the performance of a genetic algorithm by computing RMSE and visualizing results across generations.

```
# Connect to Gdrive
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import seaborn as sns

import ast
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", f
```

```
# Load the data
# This notebook cleans the Data first
# gamaster

# df = pd.read_csv('/content/drive/My Drive/data/rhdcon22x/volumes_model5.csv') #model 1
#df = pd.read_csv('/content/drive/My Drive/data/model5res/volumes.csv') #model 2
#df = pd.read_csv('/content/drive/My Drive/data/gamasterres/volumes.csv') #model 1 re-do
#df = pd.read_csv('/content/drive/My Drive/data/rhdcon01/volumes.csv') #model 1 re-do

df = pd.read_csv('/content/drive/My Drive/data/rhd70/volumes.csv') #model 1

# Compute RMSE

# Filter rows keep only rows where it doesnt saty iter
df_clean = df[df['iter'] != 'iter']

# Convert numeric columns (optional, except 'Params')
numeric_cols = df_clean.columns.drop('Params')
df_clean[numeric_cols] = df_clean[numeric_cols].apply(pd.to_numeric, errors='coerce')

# Reset index
df_clean = df_clean.reset_index(drop=True)
df_clean['gen_fixed'] = (df_clean.index // 100) + 1
df_clean['Squared_Error'] = (df_clean['EDV'] - df_clean['Target_EDV'])
df_clean['lowCost'] = np.abs(df_clean['V0_cost'] - df_clean['Vol_cost'])
df_clean['RMSE'] = df_clean['Squared_Error']
df_clean['Difference_RMSE'] = df_clean['Diff']
df_clean.tail(5)
#df.columns
```

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF_e
160	160	2	[0.5386870216913726, 23.39297332691108, 22.763...	86.4	107.785334	107.610888	3.594631	199.908081	184.46	-0.0
161	161	2	[0.5304084880609783, 24.646303224488037, 0.550...	86.4	107.785334	107.546787	3.027575	182.830689	184.46	-0.1
162	162	2	[2.9639236517329435, 24.897549689555024, 7.415...	86.4	107.785334	107.600038	3.513928	184.380187	184.46	-0.1
163	163	2	[1.3579850580993653, 20.454992761748215, 15.78...	86.4	107.785334	107.564459	2.982283	198.029816	184.46	-0.0
164	164	2	[0.04957124468755153, 21.36551024840708, 10.48...	86.4	107.785334	107.561475	2.055101	182.523219	184.46	-0.1

5 rows × 24 columns

```
# prompt: which iterations have lowCost below 10 but above 0
df_filtered = df_clean[(df_clean['lowCost'] > 0) & (df_clean['lowCost'] < 100 )]
df_filtered
```

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF_e
0	0	1	[0.451502, 1.410922, 13.379407, 4.99407, 6.601...	86.4	107.785334	107.575605	2.200000	201.426092	184.46	-0.0
1	1	1	[1.5313875827474859, 20.451280034700595, 28.30...	86.4	107.785334	107.568921	2.963263	202.303569	184.46	-0.0
2	2	1	[2.382793443549767, 11.338420704925193, 10.012...	86.4	107.785334	107.586720	2.280405	189.827692	184.46	-0.0
3	3	1	[2.585936999077126, 7.537730557594855, 1.77385...	86.4	107.785334	107.554520	2.512441	189.629454	184.46	-0.1
4	4	1	[0.19391027261326824, 6.8256772479831245, 16.4...	86.4	107.785334	107.566392	3.496684	197.574801	184.46	-0.0
...	...	...	...	...	...	...	...	...	...	
160	160	2	[0.5386870216913726, 23.39297332691108, 22.763...	86.4	107.785334	107.610888	3.594631	199.908081	184.46	-0.0
161	161	2	[0.5304084880609783, 24.646303224488037, 0.550...	86.4	107.785334	107.546787	3.027575	182.830689	184.46	-0.1
162	162	2	[2.9639236517329435, 24.897549689555024, 7.415...	86.4	107.785334	107.600038	3.513928	184.380187	184.46	-0.1
163	163	2	[1.3579850580993653, 20.454992761748215, 15.78...	86.4	107.785334	107.564459	2.982283	198.029816	184.46	-0.0
164	164	2	[0.04957124468755153, 21.36551024840708, 10.48...	86.4	107.785334	107.561475	2.055101	182.523219	184.46	-0.1

160 rows × 24 columns

EDV Error under 1%

```
# Calculate percentage EDV error for each iteration
df_filtered['EDV_pct_error'] = ((df_filtered['EDV'] - df_filtered['Target_EDV']).abs() / df_filtered['Target

# Filter for iterations where EDV error is under 1%
edv_under_1pct = df_filtered[df_filtered['Vol_cost'] <0.05]

# Display the filtered results
edv_under_1pct
```

```
/tmp/ipykernel_1631/3179483910.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
  df_filtered['EDV_pct_error'] = ((df_filtered['EDV'] - df_filtered['Target_EDV']).abs() / df_filtered['Targe
```

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF_e
2	2	1	[2.382793443549767, 11.338420704925193, 10.012...	86.4	107.785334	107.586720	2.280405	189.827692	184.46	-0.0
3	3	1	[2.585936999077126, 7.537730557594855, 1.77385...	86.4	107.785334	107.554520	2.512441	189.629454	184.46	-0.1
8	8	1	[0.10582248371735914, 3.998599459164709, 27.25...	86.4	107.785334	107.559522	2.746227	189.447800	184.46	-0.1
9	9	1	[1.4375134995058403, 28.198439428787413, 27.39...	86.4	107.785334	107.556011	2.302833	177.554430	184.46	-0.1
11	11	1	[0.34804118379995197, 20.497751725663857, 3.62...	86.4	107.785334	107.547109	2.291276	185.236618	184.46	-0.1
...	...	...	...	...	...	...	...	...	...	
158	158	2	[0.4247646269584708, 27.139433344513066, 2.738...	86.4	107.785334	107.530388	3.571106	189.032113	184.46	-0.1
159	159	2	[1.409771190615621, 16.245715112101696, 9.0968...	86.4	107.785334	107.566606	3.113393	191.388087	184.46	-0.0
161	161	2	[0.5304084880609783, 24.646303224488037, 0.550...	86.4	107.785334	107.546787	3.027575	182.830689	184.46	-0.1
162	162	2	[2.9639236517329435, 24.897549689555024, 7.415...	86.4	107.785334	107.600038	3.513928	184.380187	184.46	-0.1
164	164	2	[0.04957124468755153, 21.36551024840708, 10.48...	86.4	107.785334	107.561475	2.055101	182.523219	184.46	-0.1

95 rows × 25 columns

PHYS ERROR UNDER 1

```
# Calculate percentage EDV error for each iteration
df_filtered['EDV_pct_error'] = ((df_filtered['EDV'] - df_filtered['Target_EDV']).abs() / df_filtered['Target

# Filter for iterations where EDV error is under 1%
phys_under_1pct = df_filtered[df_filtered['Phys_rmse'] < 2]
```

```
# Display the filtered results
phys_under_1pct
```

```
/tmp/ipykernel_1631/2877156990.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
  df_filtered['EDV_pct_error'] = ((df_filtered['EDV'] - df_filtered['Target_EDV']).abs() / df_filtered['Tar
```

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF
8	8	1	[0.10582248371735914, 3.998599459164709, 27.25...	86.4	107.785334	107.559522	2.746227	189.447800	184.46	-0.
13	13	1	[0.13005189476188994, 9.80761975728273, 8.5828...	86.4	107.785334	107.556440	2.739866	190.002842	184.46	-0.
14	14	1	[1.2283968615062038, 17.755657632638314, 25.75...	86.4	107.785334	107.559936	2.220444	188.393689	184.46	-0.
40	40	1	[2.796988534380829, 8.29025829237506, 26.76246...	86.4	107.785334	107.553733	2.769546	187.547284	184.46	-0.
45	45	1	[2.9639236517329435, 24.897549689555024, 7.415...	86.4	107.785334	107.598288	2.963410	180.466478	184.46	-0.
53	53	1	[2.7091352145199425, 21.24544540833708, 5.7074...	86.4	107.785334	107.566838	2.160253	181.372863	184.46	-0.
54	54	1	[1.960248780386961, 8.442901821996891, 20.6623...	86.4	107.785334	107.539031	2.687673	193.370413	184.46	-0.
58	58	1	[0.1323255222505833, 27.980084252258056, 22.66...	86.4	107.785334	107.606022	2.174257	179.371483	184.46	-0
63	63	1	[1.653583227139559, 13.615161404646559, 28.268...	86.4	107.785334	107.593244	2.684415	199.420376	184.46	-0.
64	64	1	[0.7538441091860053, 29.88370346064237, 5.9221...	86.4	107.785334	107.575402	2.295399	186.696736	184.46	-0.
65	65	1	[0.6366289286600392, 8.585110798499159, 9.9418...	86.4	107.785334	107.580836	3.409691	192.355456	184.46	-0.
71	71	1	[2.941760991273634, 4.3405082092176945, 5.2168...	86.4	107.785334	107.565920	2.343469	189.305632	184.46	-0.
76	76	1	[0.5304084880609783, 24.646303224488037, 0.550...	86.4	107.785334	107.519852	3.096129	182.744503	184.46	-0.
84	84	1	[1.7085575079726971, 21.697974470882617, 14.98...	86.4	107.785334	107.563726	3.131287	194.786814	184.46	-0.

```
df_filtered.columns
```

```
Index(['iter', 'gen', 'Params', 'ESV_Clinical', 'ESV_Sim', 'V0_simulation',
       'EDP', 'EDV', 'Target_EDV', 'EF_error', 'Phys_rmse', 'Diff', 'Vol_cost',
       'Strain_cost', 'V0_cost', 'ESV_cost', 'loss_l', 'loss_c', 'loss_r',
       'gen_fixed', 'Squared_Error', 'lowCost', 'RMSE', 'Difference_RMSE',
       'EDV_pct_error'],
      dtype='object')
```

90	90	1	[1.3679850580993653, 20.464992767748215, 15.78...	86.4	107.785334	107.561857	2.593293	189.097880	184.46	-0.
98	98	1	[0.4758794213392763, 4.2665087961825146, 12.7...	86.4	107.785334	107.581486	2.072912	185.755279	184.46	-0
100	100	2	[0.1323255222505833, 27.980084252258056, 22.66...	86.4	107.785334	107.606022	2.174257	179.371483	184.46	-0

EF ERROR under 1%

```
# Calculate percentage EDV error for each iteration
```

```
df_filtered['EDV_pct_error'] = ((df_filtered['EDV'] - df_filtered['Target_EDV']).abs() / df_filtered['Target

# Filter for iterations where EDV error is under 1%
edv_under_1pct = df_filtered[df_filtered['EF_error'] < 1]

# Display the filtered results
edv_under_1pct.tail(20)
```

106	106	2	[2.8296280626371937, 12.601852858525872, 16.93...	86.4	107.785334	107.567734	2.963410	185.078304	184.46	-0
114	114	2	[2.163352438333684, 9.80761975728273, 8.582826...	86.4	107.785334	107.557077	3.407778	193.265492	184.46	-0.
117	117	2	[2.0432926412881858, 24.856143442826514, 19.45...	86.4	107.785334	107.566510	2.629678	184.759459	184.46	-0
119	119	2	[1.568589903758931, 25.921591770267476, 23.218...	86.4	107.785334	107.588651	2.776782	189.919183	184.46	-0.
120	120	2	[2.4959237377168257, 25.617510934124173, 11.28...	86.4	107.785334	107.572400	2.593293	193.737353	184.46	-0.
123	123	2	[0.10582248371735914, 3.998599459164709, 27.25...	86.4	107.785334	107.579162	2.629678	186.752067	184.46	-0.
126	126	2	[2.0432926412881858, 2.3757301540954265, 5.015...	86.4	107.785334	107.601982	2.377890	196.924867	184.46	-0.
129	129	2	[1.0225651039576886, 2.309335079103471, 28.488...	86.4	107.785334	107.573685	2.343469	189.836911	184.46	-0.
145	145	2	[0.13005189476188994, 9.80761975728273, 8.5828...	86.4	107.785334	107.553003	2.220444	192.917375	184.46	-0.
147	147	2	[1.5313875827474859, 20.451280034700595, 28.30...	86.4	107.785334	107.601907	3.027575	189.909072	184.46	-0.
150	150	2	[0.13005189476188994, 9.80761975728273, 8.5828...	86.4	107.785334	107.536153	2.687673	195.900205	184.46	-0
152	152	2	[0.5386870216913726, 23.39297332691108, 22.763...	86.4	107.785334	107.540943	2.996471	189.523967	184.46	-0.
161	161	2	[0.5304084880609783, 24.646303224488037, 0.550...	86.4	107.785334	107.546787	3.027575	182.830689	184.46	-0

34 rows × 25 columns

```
/tmp/ipykernel_1631/1012165989.py:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html
  df_filtered['EDV_pct_error'] = ((df_filtered['EDV'] - df_filtered['Target_EDV']).abs() / df_filtered['Tar
```

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF_
144	144	2	[2.4329892814242613, 21.504127359857062, 22.41...	86.4	107.785334	107.591065	2.107581	180.896700	184.46	-0.

Pareto Optimality check

```
pareto_points = []
for i, row_i in df_filtered.iterrows():
    dominated = False
    for j, row_j in df_filtered.iterrows():
        if (row_j['Phys_rmse'] <= row_i['Phys_rmse']) and (row_j['Vol_cost'] <= row_i['Vol_cost']) and \
           ((row_j['Phys_rmse'] < row_i['Phys_rmse']) or (row_j['Vol_cost'] < row_i['Vol_cost'])):
            dominated = True
            break
    if not dominated:
        pareto_points.append(row_i)

pareto_df = pd.DataFrame(pareto_points)
pareto_df
```

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF_er
87	87	1	[0.9367911164001519, 19.320322193588673, 4.546...	86.4	107.785334	107.494504	2.014576	184.579910	184.46	-0.11
152	152	2	[0.5388702169137 23.392973826911	86.4	107.785334	107.540943	2.996471	189.523967	184.46	-0.
117	117	2	[2.0432964128818 24.856143442826514, 22.763...	86.4	107.785334	107.566510	2.629678	184.759459	184.46	-0.11
162	162	2	[2.9639236517329435, 24.897549689555024, 7.415...	86.4	107.785334	107.600038	3.513928	184.380187	184.46	-0.11
154	154	2	[1.2283968615062038, 17.755657632638314, 25.75...	86.4	107.785334	107.577280	2.982283	203.548849	184.46	-0
156	156	2	[0.4863798540508707 12.972012113052706, 5.23...	86.4	107.785334	107.549919	3.096129	183.179567	184.46	-0.
157	157	2	[1.960848780386961, 8.442901821996891, 20.6623...	86.4	107.785334	107.573975	3.594631	199.759752	184.46	-0
158	158	2	[0.4241648286584703, 27.139433344543066, 2.738...	86.4	107.785334	107.530388	3.571106	189.032113	184.46	-0.

3 rows × 25 columns

```
pareto_df['iter'].tolist()
```

```
[87, 117, 162]
```

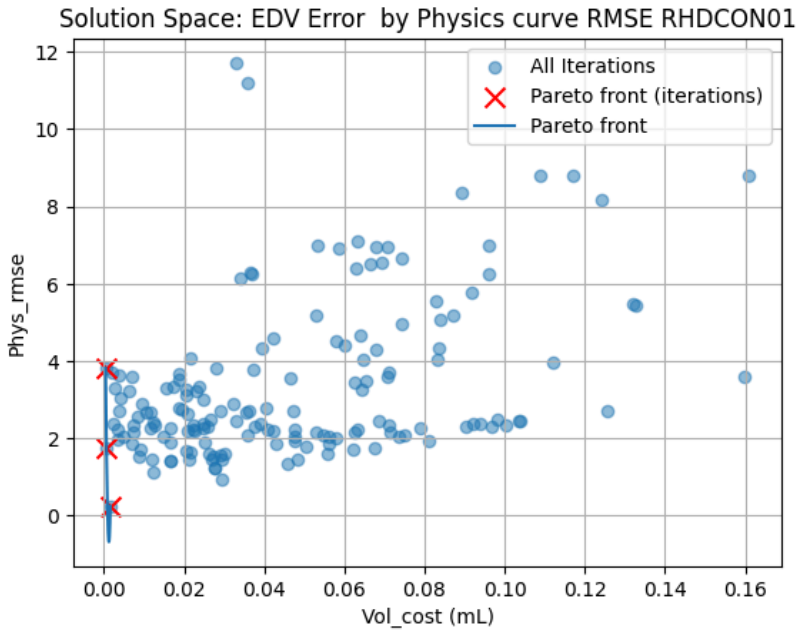
PLOT PARETO FRONT

"We evaluate Pareto optimality using Phys_rmse and EDV error to explore all feasible solutions. The Pareto front allows us to identify non-dominated solutions and then choose the most physiologically and clinically appropriate result."

```
x = pareto_df['Vol_cost'].values
y = pareto_df['Phys_rmse'].values
coeffs = np.polyfit(x, y, deg=2)         # [a, b, c]
poly = np.poly1d(coeffs)

# Optional: print the fitted equation
a, b, c = coeffs
print(f"Fitted curve: Phys_rmse ≈ {a:.6f} * Vol_cost^2 + {b:.6f} * Vol_cost + {c:.6f}")

# --- Make a smooth curve for plotting ---
x_curve = np.linspace(x.min(), x.max(), 200)
y_curve = poly(x_curve)

plt.figure()
plt.scatter(df_filtered['Vol_cost'], df_filtered['Phys_rmse'], label='All Iterations', alpha=0.5)
```

```
plt.scatter(pareto_df['Vol_cost'], pareto_df['Phys_rmse'], color='red', marker="x" ,label='Pareto front (ite
plt.plot(x_curve, y_curve, label='Pareto front')

#for idx, row in pareto_df.iterrows():
#     plt.text(row['Vol_cost'] + 0.001, row['Phys_rmse'] + 0.01,  # slight shift
#              str(int(row['iter'])), fontsize=8)
"""for idx, row in pareto_df.iterrows():
    if int(row['iter']) == 152:   # only iteration 46
        plt.text(row['Vol_cost'] + 0.001, row['Phys_rmse'] + 0.001,
                 "152", fontsize=10, fontweight='bold', color='black')"""

plt.xlabel('Vol_cost (mL)')
plt.ylabel('Phys_rmse')
plt.title('Solution Space: EDV Error  by Physics curve RMSE RHDCON01')
plt.grid(True)
plt.legend()
plt.show()
```

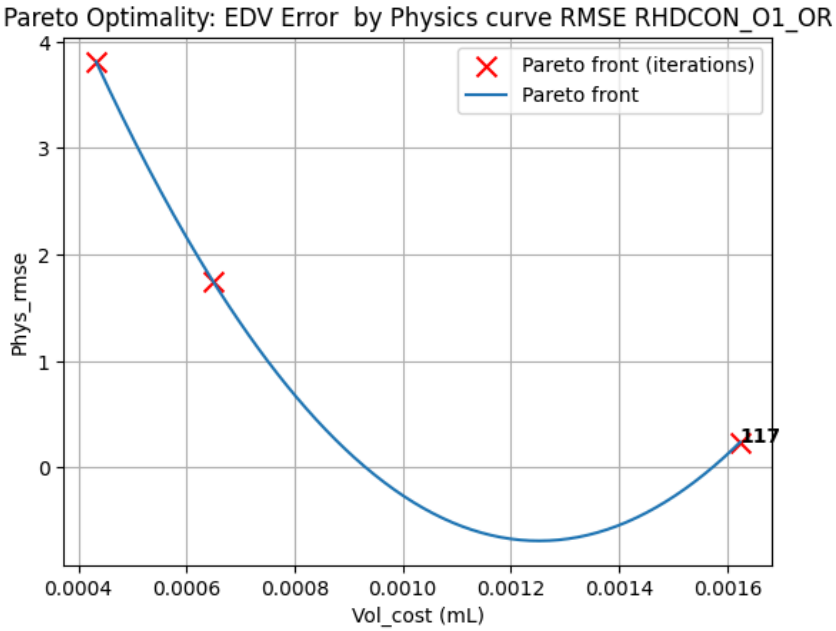
```
Fitted curve: Phys_rmse ≈ 6710611.008862 * Vol_cost^2 + -16801.286719 * Vol_cost + 9.826199
```



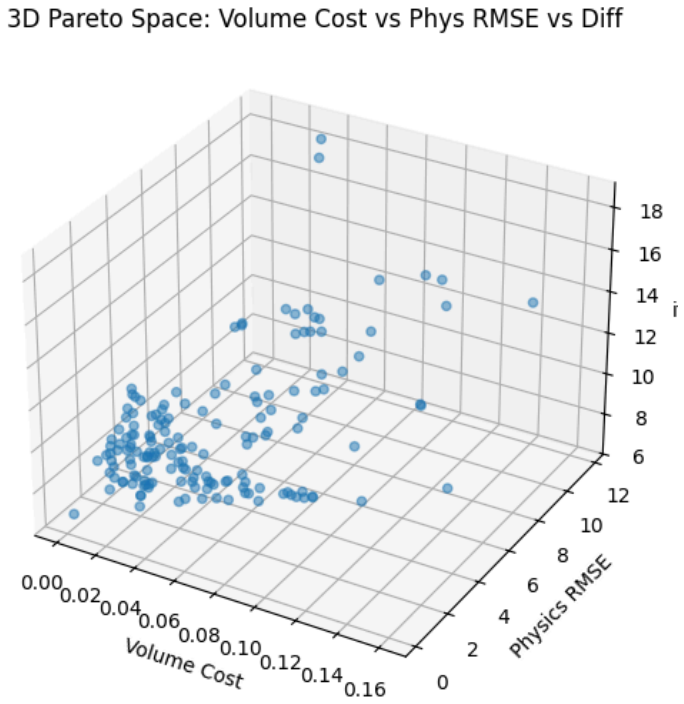

```
x = pareto_df['Vol_cost'].values
y = pareto_df['Phys_rmse'].values
coeffs = np.polyfit(x, y, deg=2)         # [a, b, c]
poly = np.poly1d(coeffs)

# Optional: print the fitted equation
a, b, c = coeffs
print(f"Fitted curve: Phys_rmse ≈ {a:.6f} * Vol_cost^2 + {b:.6f} * Vol_cost + {c:.6f}")

# --- Make a smooth curve for plotting ---
x_curve = np.linspace(x.min(), x.max(), 200)
y_curve = poly(x_curve)

plt.figure()
#plt.scatter(df_filtered['Vol_cost'], df_filtered['Phys_rmse'], label='All Iterations', alpha=0.5)
plt.scatter(pareto_df['Vol_cost'], pareto_df['Phys_rmse'], color='red', marker="x" ,label='Pareto front (ite
plt.plot(x_curve, y_curve, label='Pareto front')

#for idx, row in pareto_df.iterrows():
#     plt.text(row['Vol_cost'] + 0.001, row['Phys_rmse'] + 0.01,  # slight shift
#              str(int(row['iter'])), fontsize=8)
for idx, row in pareto_df.iterrows():
    if int(row['iter']) == 117:   # only iteration 46
        plt.text(row['Vol_cost'] + 0.000, row['Phys_rmse'] + 0.000,
                 "117", fontsize=10, fontweight='bold', color='black')

plt.xlabel('Vol_cost (mL)')
plt.ylabel('Phys_rmse')
```

```
plt.title('Pareto Optimality: EDV Error  by Physics curve RMSE RHDCON_O1_OR')
plt.grid(True)
plt.legend()
plt.show()
```

```
Fitted curve: Phys_rmse ≈ 6710611.008862 * Vol_cost^2 + -16801.286719 * Vol_cost + 9.826199
```




```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401 (needed for 3D)

# Create figure and 3D axis
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

# 3D scatter plot
ax.scatter(
    df_filtered['Vol_cost'],
    df_filtered['Phys_rmse'],
    df_filtered['Diff'],
    alpha=0.5
)

# Axis labels
ax.set_xlabel('Volume Cost')
ax.set_ylabel('Physics RMSE')
ax.set_zlabel('iter')

# Optional title
ax.set_title('3D Pareto Space: Volume Cost vs Phys RMSE vs Diff')

plt.show()
```




```
df_filtered.columns
```

```
Index(['iter', 'gen', 'Params', 'ESV_Clinical', 'ESV_Sim', 'V0_simulation',
       'EDV', 'Target_EDV', 'EF_error', 'Phys_rmse', 'klotz_rmse', 'Diff',
       'Vol_cost', 'Strain_cost', 'V0_cost', 'ESV_cost', 'loss_l', 'loss_c',
       'loss_r', 'gen_fixed', 'Squared_Error', 'lowCost', 'RMSE',
       'Difference_RMSE', 'EDV_pct_error'],
      dtype='object')
```

Double-click (or enter) to edit

PARAMETER RANGES

```
lowest_diff_row = df_filtered.loc[df_filtered['Diff'].idxmin()]
print("Row with the lowest 'Diff' value:")
display(lowest_diff_row)

print("\nRow with index 162:")
display(df_filtered.loc[0])

lowest_diff_new=df_filtered.loc[0]
```

```
Row with the lowest 'Diff' value:
```

	117
iter	117
gen	2
Params	[2.0432926412881858, 24.856143442826514, 19.45...
ESV_Clinical	86.4
ESV_Sim	107.785334
V0_simulation	107.56651
EDP	2.629678
EDV	184.759459
Target_EDV	184.46
EF_error	-0.114988
Phys_rmse	0.236504
Strain_cost	15.363493

```
#  Find lowest Diff row
lowest_diff_row = df_filtered.loc[0] #df_filtered.loc[df_filtered['Diff'].idxmin()]
lowest_diff_row
```

	0
iter	0
gen	1
Params	[0.451502, 1.410922, 13.379407, 4.99407, 6.601...
ESV_Clinical	86.4
ESV_Sim	107.785334
V0_simulation	107.575605
EDP	2.2
EDV	201.426092
Target_EDV	184.46
EF_error	-0.066717
Phys_rmse	2.365214
Diff	9.029909
Vol_cost	0.091977
Strain_cost	15.270253

	0
V0_cost	0.171068
ESV_cost	0.247515
loss_l	6.220851
loss_c	4.348928
loss_r	4.700474
gen_fixed	1
Squared_Error	16.966092
lowCost	0.079091
RMSE	16.966092
Difference_RMSE	9.029909
EDV_pct_error	9.197708
Diff	9.029909
Vol_cost	0.091977

dtype: object

dtype: object

```
lowest_diff_row['Params']
```

```
'[0.451502, 1.410922, 13.379407, 4.99407, 6.601561, 9.534021, 0.765347, 3.729085, 2.291734, 7.616221, 0.545395, 22.788196, 4.207521, 9.475181, 1.143529, 19.440434, 2.107703, 4.679104, 20.493534, 1.928557, 7.54257, 0.769893, 3.315698, 18.816445, 1.793259, 7.482189, 2.228114503, 17.616866, 9.455916, 0.416957, 1.396987, 10.487535, 0.681122, 8.093864, 25.944133, 3.047704, 1.83078, 9.651593, 2.262059, 10.611839, 0.938432, 14.563957, 24.252076, 23.165343, 6.599722, 1.043166, 1.012467, 3.892241682, 1.428799, 3.299709, 4.225084, 13.695291, 0.123292, 8.196488, 2.178148, 1.869253, 1.089105, 7.390223, 23.171408, 4.908203, 7.074441, 9.625938, 1.74881, 12.388474, 9.753782, 18.264737, 20.910081, 18.063474, 7.697476, 7.871879, 1.156753, 8.510018, 1.029367, 10.877872, 18.716099, 19.943034, 3.886991, 6.319542, 0.409328, 3.261545, 0.672925, 15.619072, 19.713467, 2.712205, 6.895324, 8.075992, 2.472121, 11.150511, 1.226237, 4.571957, 20.674904, 13.417869, 0.685051, 7.260627, 0.963804, 1.503778, 2.870574, 18.159, '
```

```
Pareto_front1 = df_filtered.loc[df_filtered['iter']==0]
Pareto_front1['Params']
```

	Params
0	[0.451502, 1.410922, 13.379407, 4.99407, 6.601...

dtype: object

dtype: object

```
Pareto_front1 = df_filtered.loc[df_filtered['iter']==0]

Pareto_front1.T
```

	0
iter	0
gen	1
Params	[0.451502, 1.410922, 13.379407, 4.99407, 6.601...
ESV_Clinical	86.4
ESV_Sim	107.785334
V0_simulation	107.575605
EDP	2.2
EDV	201.426092
Target_EDV	184.46
EF_error	-0.066717
Phys_rmse	2.365214
Diff	9.029909
Vol_cost	0.091977
Strain_cost	15.270253
V0_cost	0.171068
ESV_cost	0.247515
loss_l	6.220851
loss_c	4.348928
loss_r	4.700474
gen_fixed	1
Squared_Error	16.966092
lowCost	0.079091
RMSE	16.966092
Difference_RMSE	9.029909
EDV_pct_error	9.197708

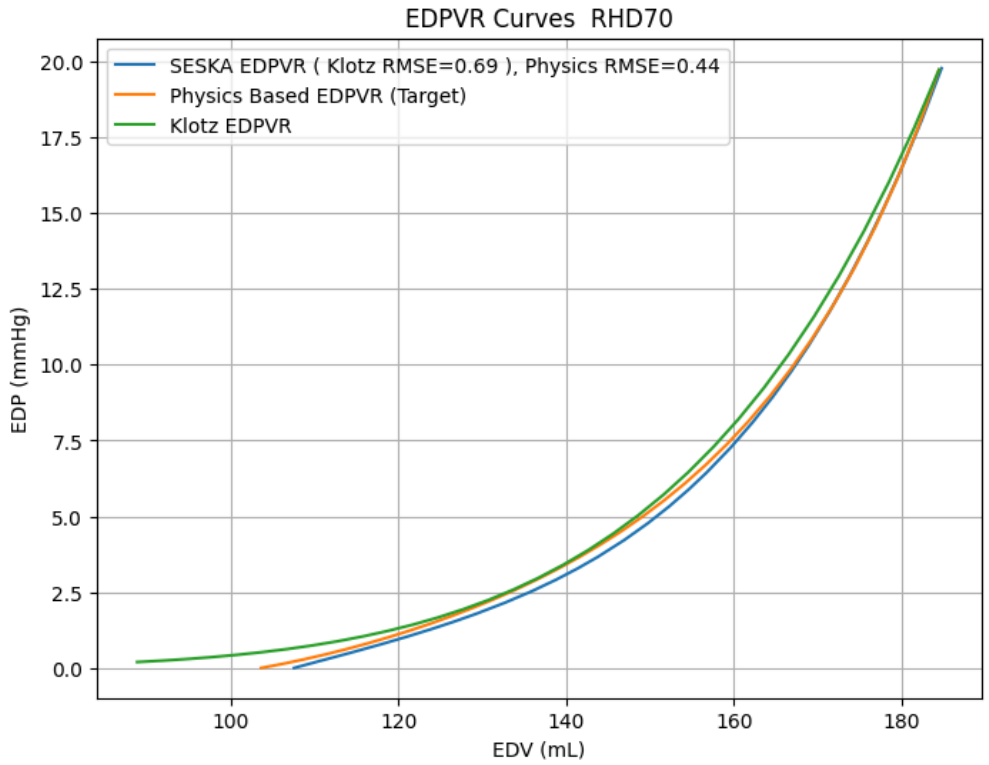
```
params_str = lowest_diff_row['Params']
params_str
```

```
len(params_str)
```

```
1316
```

CURVE RMSE for best iteration

```
import pandas as pd
import numpy as np
from scipy.interpolate import interp1d

#df = pd.read_csv('/content/drive/My Drive/data/controlscaled88res/edpvr_curves_331.csv')
#df = pd.read_csv('/content/drive/My Drive/data/baselineres/iteration_374/edpvr_curves_374.csv')
#df = pd.read_csv('/content/drive/My Drive/data/rhdcon22x/edpvr_curves_81.csv')
df = pd.read_csv('/content/drive/My Drive/data/rhd70/iteration_117/edpvr_curves_117.csv')

# Klotz reference curve
V_klotz = df['EDV_Klotz']
P_klotz = df['EDP_Klotz']

# Simulation curve (scaled to mmHg)
sim_edv = df['EDV_Simulation']
sim_edp = df['EDP_Simulation'] / 0.133

# Physics optimised curve
phys_edv = df['EDV_Optimised']
phys_edp = df['EDP_Optimised']

def calculate_rmse(P_Klotz, edp_curve, edv_range, V_klotz):
    f_interp = interp1d(edv_range, edp_curve, kind='linear', fill_value="extrapolate")
    edp_interp = f_interp(V_klotz)
    return np.sqrt(np.mean((edp_interp - P_Klotz) ** 2))

rmse_simulation = calculate_rmse(P_klotz, sim_edp, sim_edv, V_klotz)
rmse_physics = calculate_rmse(P_klotz, phys_edp, phys_edv, V_klotz)

rmse_simulation, rmse_physics

# Plot
plt.figure(figsize=(8, 6))
plt.plot(sim_edv, sim_edp,
         label=f'SESKA EDPVR ( Klotz RMSE={rmse_simulation:.2f} ), Physics RMSE={rmse_physics:.2f}',
         linestyle='-', marker='')

plt.plot(phys_edv, phys_edp,
         label=f'Physics Based EDPVR (Target) ',#(RMSE={rmse_physics:.2f} mmHg)',
         linestyle='-', marker='')

plt.plot(V_klotz, P_klotz,
         label='Klotz EDPVR',
         linestyle='-', marker='')

plt.xlabel('EDV (mL)')
plt.ylabel('EDP (mmHg)')
plt.title('EDPVR Curves  RHD70')
plt.legend()
plt.grid(True)
plt.show()
```




```
params_str = lowest_diff_row['Params']
params_str
```

```
'[0.451502, 1.410922, 13.379407, 4.99407, 6.601561, 9.534021, 0.765347, 3.729085, 2.291734, 7.616221, 0.5453
95, 22.788196, 4.207521, 9.475181, 1.143529, 19.440434, 2.107703, 4.679104, 20.493534, 1.928557, 7.54257, 0.
769893, 2.315698, 18.816445, 1.793259, 7.482189, 22.114503, 17.616866, 9.455916, 0.416957, 1.396987, 10.4875
35, 0.681122, 8.093864, 25.944133, 3.047704, 1.83078, 9.651593, 2.262059, 10.611839, 0.938432, 14.563957, 2
4.252976, 23.165343, 6.599722, 1.043166, 1.012467, 2.741682, 1.428799, 3.299709, 4.225084, 13.695291, 0.1232
92, 8.196488, 2.178148, 1.869253, 1.089105, 7.390223, 23.171408, 4.908203, 7.074441, 9.625938, 1.74881, 12.3
88474, 0.753782, 18.264737, 20.910081, 18.063474, 7.897476, 7.871879, 1.156753, 8.510018, 1.029367, 10.87787
2, 18.716099, 19.943034, 3.886991, 6.319542, 0.409328, 3.261545, 0.672925, 15.619072, 19.713467, 2.712205,
6.895324, 8.075992, 2.472121, 11.150511, 1.226237, 4.571957, 20.674904, 13.417869, 0.685051, 7.260627, 0.963
804, 1.503778, 2.870574, 18.159, '
```

```
# Find lowest Diff row
#lowest_diff_row = lowest_diff_new# RHDCon01 only
lowest_diff_row= df_filtered.loc[0] #df_filtered['Diff'].idxmin()]
# Extract parameter string and convert to list

params_str = lowest_diff_row['Params']
lowestgen = lowest_diff_row['gen']

params_list = [float(x) for x in params_str.strip('[]').split(',')]

# Separate EDP and segment parameters
edp = params_list[-1]                  # Last value is EDP
param_values = np.array(params_list[:-1])  # First 128 values are segment params

# Reshape into 16 segments × 8 parameters
param_matrix = param_values.reshape(16, 8)

# Create DataFrame for parameters
columns = ['a', 'b', 'af', 'bf', 'as', 'bs', 'afs', 'bfs']
df_params = pd.DataFrame(param_matrix, columns=columns)
df_params.index = [f"Segment {i+1}" for i in range(16)]

#Compute min and max ranges for each parameter
ranges = df_params.agg(['min', 'max'])

#Display results
print("End-Diastolic Pressure (EDP):", edp)
print("\nParameter Ranges (across 16 segments):")
```

```
print("Min Diff:",df_filtered['Diff'].min())
print("Lowest Gen:",lowestgen)
print("Lowest iteration:",lowest_diff_row['iter'])
print(lowest_diff_row)

#print(ranges)
#print(edp)
#display(df_params)
#ranges

display(ranges)
```

```
End-Diastolic Pressure (EDP): 2.2

Parameter Ranges (across 16 segments):
Min Diff: 6.792223321034541
Lowest Gen: 1
Lowest iteration: 0
iter                                                          0
gen                                                           1
Params             [0.451502, 1.410922, 13.379407, 4.99407, 6.601...
ESV_Clinical                                               86.4
ESV_Sim                                                107.785334
V0_simulation                                          107.575605
EDP                                                           2.2
EDV                                                    201.426092
Target_EDV                                                184.46
EF_error                                                -0.066717
Phys_rmse                                                2.365214
Diff                                                     9.029909
Vol_cost                                                 0.091977
Strain_cost                                             15.270253
V0_cost                                                  0.171068
ESV_cost                                                 0.247515
loss_l                                                   6.220851
loss_c                                                   4.348928
loss_r                                                   4.700474
gen_fixed                                                     1
Squared_Error                                           16.966092
lowCost                                                  0.079091
RMSE                                                    16.966092
Difference_RMSE                                          9.029909
EDV_pct_error                                            9.197708
Name: 0, dtype: object
```

1 to 2 of 2 entries Filter

Copy table

CSV JSON Markdown

Copy

```
index,a,b,af,bf,as,bs,afs,bfs
min,0.333716,1.410922,0.545395,1.928557,0.123292,0.416957,0.409328,1.503778
max,2.870574,19.100237,29.098548,23.433841,9.455916,9.651593,2.829696,19.455808
```

index	a	b	af	bf	as	bs	afs	bfs
min	0.333716	1.410922	0.545395	1.928557	0.123292	0.416957	0.409328	1.503778
max	2.870574	19.100237	29.098548	23.433841	9.455916	9.651593	2.829696	19.455808

Show 25 per page

Like what you see? Visit the data table notebook to learn more about interactive tables.

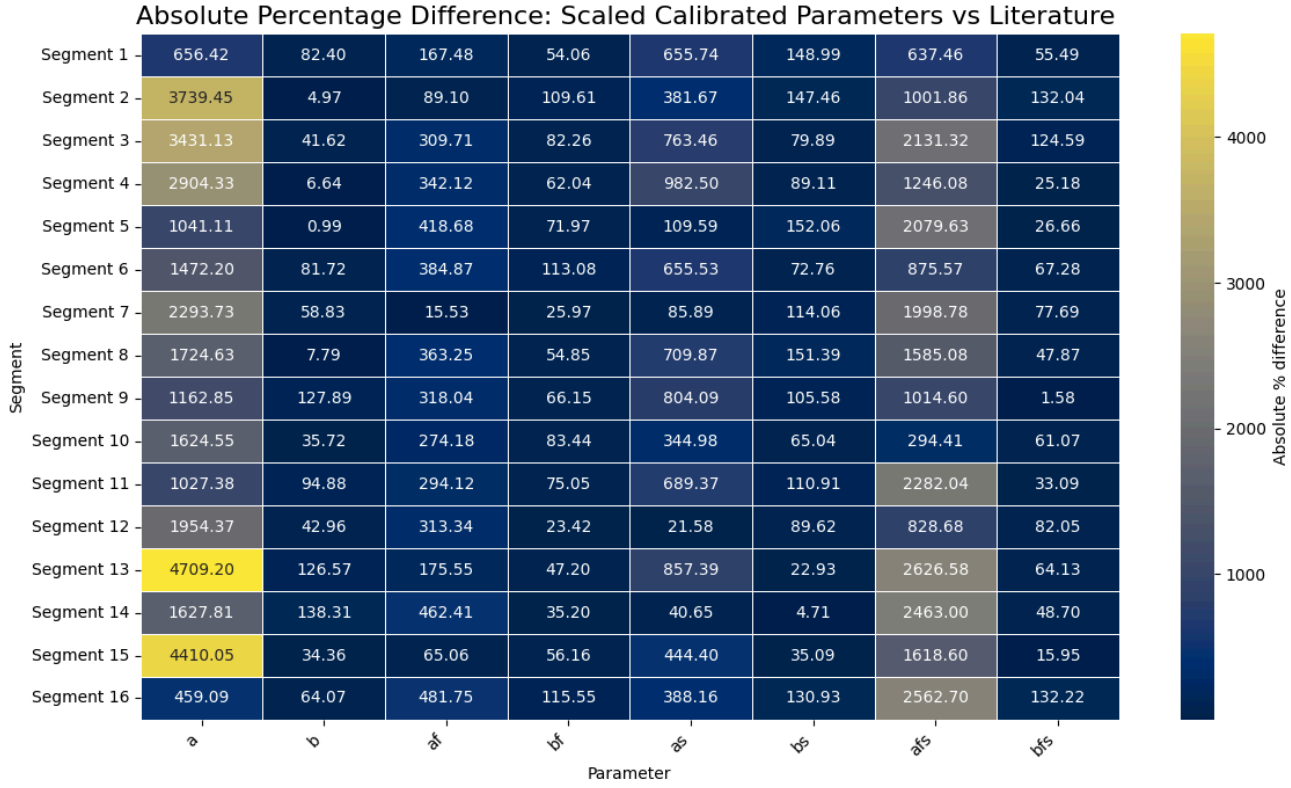
Next steps: Generate code with ranges New interactive sheet

Parameter Distribution Heatmap

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

#A_scale = 0.693389
```

```
#B_scale = 0.824769
A_scale     =  0.852751# 0.2474288661928148
# 0.852751
B_scale    =   0.809262# 1.2521354948319154 #
# 0.809262
# 0.2474288661928148, 1.2521354948319154

# Literature reference (UNSCALED)
global_values_raw = {
    'a': 0.0509,
    'b': 6.4860,
    'af': 4.2654,
    'bf': 8.7982,
    'as': 0.7449,
    'bs': 3.0987,
    'afs': 0.0885,
    'bfs': 6.7800
}

# ----------------------------
# 1) Scale calibrated values (df_params)
# ----------------------------
a_cols = ['a', 'af', 'as', 'afs']
b_cols = ['b', 'bf', 'bs', 'bfs']

df_scaled = df_params.copy()
df_scaled[a_cols] = df_scaled[a_cols] * A_scale
df_scaled[b_cols] = df_scaled[b_cols] * B_scale

# ----------------------------
# 2) Absolute Percentage Error (APE) vs Literature
# ----------------------------
lit_series = pd.Series(global_values_raw)

difference_pct = (
    df_scaled.sub(lit_series, axis=1).abs()
             .div(lit_series.abs(), axis=1)
             * 100.0
)

diff_pct_round = difference_pct.round(2)

print("--- APE: Scaled Calibrated Params vs Literature (%) ---")
print(diff_pct_round)
print("\n")

# ----------------------------
# 3) Heatmap
# ----------------------------
fig, ax = plt.subplots(figsize=(12, 7))

sns.heatmap(
    difference_pct,
    annot=diff_pct_round,
    fmt=".2f",
    cmap="cividis",
    linewidths=0.5,
    ax=ax,
    cbar_kws={'label': 'Absolute % difference'}
)

ax.set_title("Absolute Percentage Difference: Scaled Calibrated Parameters vs Literature ", fontsize=16)
ax.set_xlabel("Parameter")
ax.set_ylabel("Segment")

plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
plt.setp(ax.get_yticklabels(), rotation=0)

plt.tight_layout()
plt.savefig("seaborn_ape_heatmap_scaled_vs_lit.png", dpi=300)
print("Saved: seaborn_ape_heatmap_scaled_vs_lit.png")
```

```
--- APE: Scaled Calibrated Params vs Literature (%) ---
                  a       b      af      bf      as      bs      afs     bfs
Segment 1    656.42   82.40  167.48   54.06  655.74  148.99   637.46   55.49
Segment 2   3739.45    4.97   89.10  109.61  381.67  147.46  1001.86  132.04
Segment 3   3431.13   41.62  309.71   82.26  763.46   79.89  2131.32  124.59
Segment 4   2904.33    6.64  342.12   62.04  982.50   89.11  1246.08   25.18
Segment 5   1041.11    0.99  418.68   71.97  109.59  152.06  2079.63   26.66
Segment 6   1472.20   81.72  384.87  113.08  655.53   72.76   875.57   67.28
Segment 7   2293.73   58.83   15.53   25.97   85.89  114.06  1998.78   77.69
Segment 8   1724.63    7.79  363.25   54.85  709.87  151.39  1585.08   47.87
Segment 9   1162.85  127.89  318.04   66.15  804.09  105.58  1014.60    1.58
Segment 10  1624.55   35.72  274.18   83.44  344.98   65.04   294.41   61.07
Segment 11  1027.38   94.88  294.12   75.05  689.37  110.91  2282.04   33.09
Segment 12  1954.37   42.96  313.34   23.42   21.58   89.62   828.68   82.05
Segment 13  4709.20  126.57  175.55   47.20  857.39   22.93  2626.58   64.13
Segment 14  1627.81  138.31  462.41   35.20   40.65    4.71  2463.00   48.70
Segment 15  4410.05   34.36   65.06   56.16  444.40   35.09  1618.60   15.95
Segment 16   459.09   64.07  481.75  115.55  388.16  130.93  2562.70  132.22


Saved: seaborn_ape_heatmap_scaled_vs_lit.png
```




```
# Show top 3 rows with Diff close to min Diff

# Find the minimum value in the 'Diff' column
min_diff = df_clean['Diff'].min()

# Calculate the absolute difference from the minimum for each row
df_clean['Abs_Diff_from_Min'] = abs(df_clean['Diff'] - min_diff)

# Sort the DataFrame by the absolute difference and get the top 3 rows
top_3_closest_to_min = df_clean.sort_values(by='Abs_Diff_from_Min').head(5)

# Display the result
print("Top 3 rows with Diff closest to the minimum Diff:")
display(top_3_closest_to_min)

# Drop the temporary 'Abs_Diff_from_Min' column if you don't need it
df_clean = df_clean.drop(columns=['Abs_Diff_from_Min'])
```

Top 3 rows with Diff closest to the minimum Diff:

	iter	gen	Params	ESV_Clinical	ESV_Sim	V0_simulation	EDP	EDV	Target_EDV	EF_e
117	117	2	[2.0432926412881858, 24.856143442826514, 19.45...	86.4	107.785334	107.566510	2.629678	184.759459	184.46	-0.1
119	119	2	[1.568589903758931, 25.921591770267476, 23.218...	86.4	107.785334	107.588651	2.776782	189.919183	184.46	-0.0
123	123	2	[0.10582248371735914, 3.998599459164709, 27.25...	86.4	107.785334	107.579162	2.629678	186.752067	184.46	-0.1
58	58	1	[0.1323255222505833, 27.980084252258056, 22.66...	86.4	107.785334	107.606022	2.174257	179.371483	184.46	-0.1
100	100	2	[0.1323255222505833, 27.980084252258056, 22.66...	86.4	107.785334	107.606022	2.174257	179.371483	184.46	-0.1

5 rows × 25 columns

```
df_clean.columns
```

```
Index(['iter', 'gen', 'Params', 'ESV_Clinical', 'ESV_Sim', 'V0_simulation',
       'EDP', 'EDV', 'Target_EDV', 'EF_error', 'Phys_rmse', 'Diff', 'Vol_cost',
       'Strain_cost', 'V0_cost', 'ESV_cost', 'loss_l', 'loss_c', 'loss_r',
       'gen_fixed', 'Squared_Error', 'lowCost', 'RMSE', 'Difference_RMSE'],
      dtype='object')
```

```
# prompt: create a df of strains from df_longsstrain = pd.read_csv('/content/drive/My Drive/data/garesults/s
#df = pd.read_csv('/content/drive/My Drive/data/garesults/simulatedstrains/model5gc/iteration_5
"""df_radial = pd.read_csv('/content/drive/My Drive/data/baselineres/iteration_374/max_radial_strain.csv')
df_long = pd.read_csv('/content/drive/My Drive/data/baselineres/iteration_374/max_long_strain.csv')
df_circ = pd.read_csv('/content/drive/My Drive/data/baselineres/iteration_374/max_circ_strain.csv')"""

df_radial = pd.read_csv('/content/drive/My Drive/data/rhd70/iteration_117/max_radial_strain.csv')
df_long = pd.read_csv('/content/drive/My Drive/data/rhd70/iteration_117/max_long_strain.csv')
df_circ = pd.read_csv('/content/drive/My Drive/data/rhd70/iteration_117/max_circ_strain.csv')

# Assuming the common column is named 'Common_Column' in all files
# Replace 'Common_Column' with the actual name of your common column
#df_strains = pd.merge(df_radial, df_long, on='Segment', how='inner')
#df_strains = pd.merge(df_strains, df_circ, on='Segnent', how='inner')

#print(df_strains.head())

sim_strain_df = pd.DataFrame({'radial_strain': df_radial['Max_radial_Strain'],
                              'circ_strain':   df_circ['Max_circ_Strain'],
                              'long_strain':   df_long['Max_long_Strain']})
```

```
print(pd.DataFrame(sim_strain_df))
```

```
    radial_strain  circ_strain  long_strain
0        0.161569     0.161697     0.015657
1        0.162269     0.178059     0.012556
2        0.179416     0.226116     0.007603
3        0.156928     0.175961     0.015835
4        0.162200     0.194373     0.022713
5        0.200090     0.235751     0.018483
6        0.130897     0.148644     0.003001
7        0.108564     0.117195     0.005022
8        0.163406     0.180991     0.016665
9        0.129432     0.143439     0.016140
10       0.096624     0.140880     0.013844
11       0.122696     0.157749     0.014152
12       0.078151     0.140777     0.038588
13       0.051304     0.077851     0.016223
```

```
14       0.078944     0.161420     0.033308
15       0.069113     0.139324     0.049571
```

```
#clinical_strains =  pd.read_csv('/content/drive/My Drive/data/controlscaled88res/absolute_strains.csv')
clinical_strains =  pd.read_csv('/content/drive/My Drive/data/rhd70/absolute_strains.csv')

clinical_strains['absolute_strain'] = clinical_strains['Absolute_Relative_Strain']/100
clinical_strains.head()
```

	ES	ED	Absolute_Relative_Strain	absolute_strain
0	-23.061	-0.993	16.035	0.16035
1	-15.232	-1.140	12.313	0.12313
2	-7.662	1.273	5.326	0.05326
3	-15.078	-0.300	12.542	0.12542
4	-22.647	-0.938	18.820	0.18820

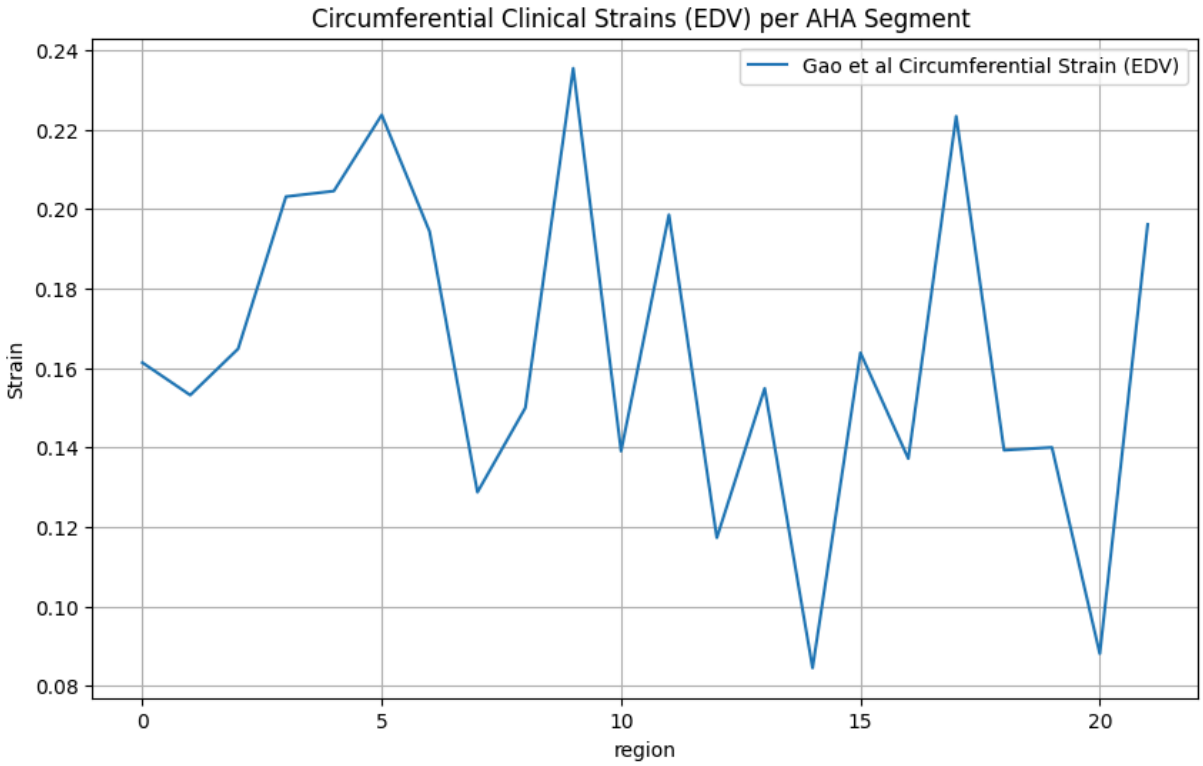
Next steps: Generate code with clinical_strains | New interactive sheet

```
gaostrains = pd.read_csv('/content/drive/MyDrive/data/gaostrains.csv', header=None)
gaostrains.columns = ['region', 'strain']
gaostrains.head()
```

	region	strain
0	2.657652	0.161357
1	3.623926	0.153185
2	4.586754	0.164831
3	5.571025	0.203129
4	6.509558	0.204524

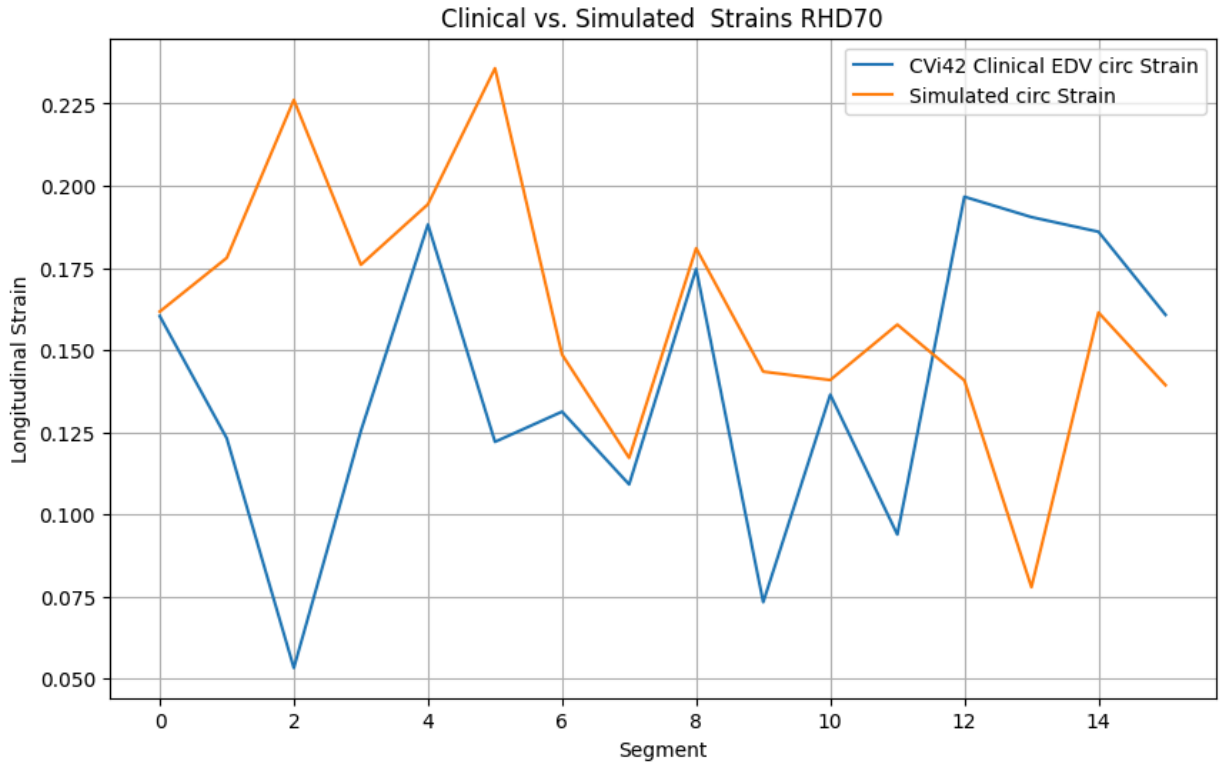
Next steps: Generate code with gaostrains | New interactive sheet

```
plt.figure(figsize=(10, 6))
#plt.plot(clinical_strains.index, clinical_strains['edv_rad_strain'], label='Radial Strain (EDV)')
#plt.plot(clinical_strains.index, clinical_strains['edv_circ_strain']/100, label='Circumferential Strain (EI
#plt.plot(clinical_strains.index, clinical_strains['edv_long_Strain'], label='Longitudinal Strain (EDV)')
plt.plot(gaostrains.index,gaostrains['strain'], label='Gao et al Circumferential Strain (EDV)')

plt.xlabel('region')
plt.ylabel('Strain')
plt.title('Circumferential Clinical Strains (EDV) per AHA Segment ')
plt.legend()
plt.grid(True)
plt.show()
```



PLOT LONGITUDINAL STRAIN ONLY

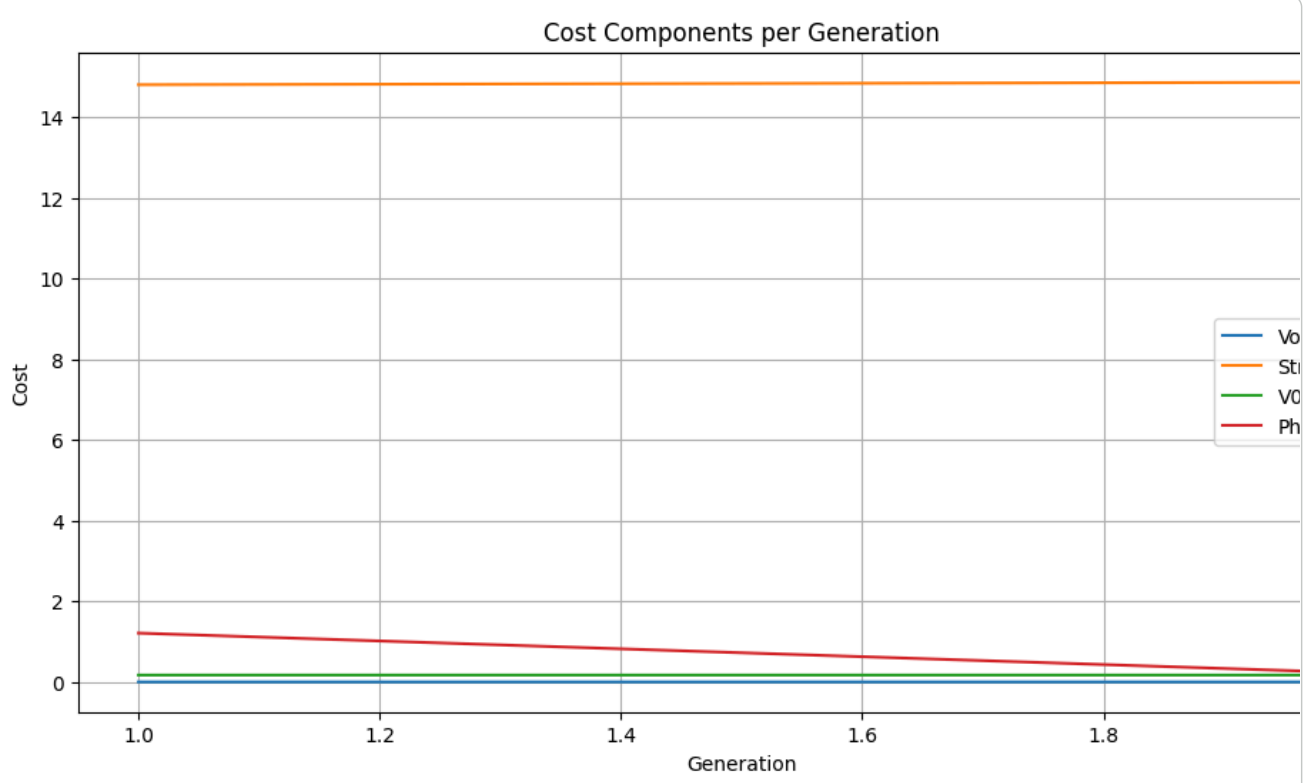
```
plt.figure(figsize=(10, 6))

#plt.plot(clinical_strains.index, clinical_strains['edv_long_Strain'], label='Clinical Longitudinal Strain')
#plt.plot(df_long.index, df_long['Max_long_Strain'], label='Simulated Longitudinal Strain')

#plt.plot(clinical_strains.index, clinical_strains['edv_long_Strain'], label='Clinical Longitudinal Strain')
#plt.plot(ground_truth_strain.index, ground_truth_strain['radial_strain'], label='Simulated radial Strain')
#plt.plot(sim_strain_df.index, sim_strain_df['long_strain']*50, label='Simulated long Strain')

plt.plot(clinical_strains.index, clinical_strains['absolute_strain'], label='CVi42 Clinical EDV circ Strain
plt.plot(sim_strain_df.index, sim_strain_df['circ_strain'], label='Simulated circ Strain')#6
#plt.plot(gaostrains.index,gaostrains['strain'], label='Gao et al Circumferential Strain (EDV)')

#plt.plot(clinical_strains.index, clinical_strains['edv_rad_strain'], label='Clinical rad Strain')
#plt.plot(sim_strain_df.index, sim_strain_df['radial_strain']*1, label='Simulated rad Strain')#10

plt.xlabel('Segment')
plt.ylabel('Longitudinal Strain')
plt.title('Clinical vs. Simulated  Strains RHD70')
plt.legend()
plt.grid(True)
plt.show()
```




```
print(df_clean['Diff'].min())
df = df_clean
```

```
6.792223321034541
```

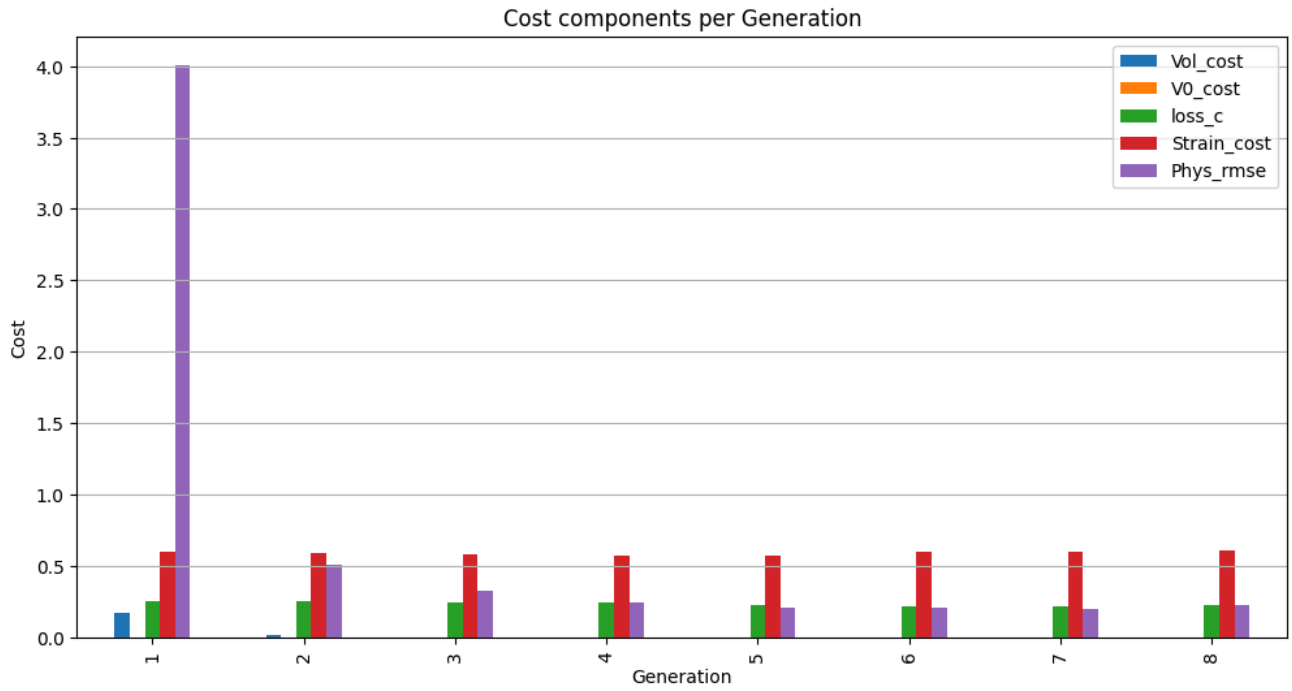
```
# Group data by generation and calculate the mean of Vol_cost, Strain_cost, and V0_cost
generation_stats = df_filtered.groupby('gen')[['Vol_cost', 'Strain_cost','loss_c','loss_r','loss_l','V0_cost

generation_stats

# Plotting the trends
plt.figure(figsize=(12, 6))
plt.plot(generation_stats.index, generation_stats['Vol_cost'], label='Vol_cost')
plt.plot(generation_stats.index, generation_stats['Strain_cost'], label='Strain_cost')
plt.plot(generation_stats.index, generation_stats['V0_cost'], label='V0_cost')
plt.plot(generation_stats.index, generation_stats['Phys_rmse'], label='Phys_rmse')
#plt.plot(generation_stats.index, generation_stats['RMSE'], label='Klotz_error')


plt.xlabel('Generation')
plt.ylabel('Cost')
plt.title('Cost Components per Generation')
plt.legend()
plt.grid(True)
plt.show()

# Print the generation_stats DataFrame for detailed numerical analysis
generation_stats
```



gen	Vol_cost	Strain_cost	loss_c	loss_r	loss_l	V0_cost	Phys_rmse
1	0.000650	14.805512	4.110443	4.545848	6.149222	0.170186	1.211166
2	0.000433	14.865646	4.143998	4.569966	6.151682	0.170368	0.236504

Next steps: Generate code with generation_stats | New interactive sheet

```
# prompt: plot as bar graph generation_stats

# Plotting as bar graph
plt.figure(figsize=(12, 6))

generation_stats[['Vol_cost','V0_cost','loss_c','Strain_cost','Phys_rmse']].plot(kind='bar', ax=plt.gca())#,
#generation_stats[['Vol_cost', 'loss_c','loss_r','loss_l','V0_cost']].plot(kind='bar', ax=plt.gca())

#generation_stats[['loss_c','loss_r','loss_l']].plot(kind='bar', ax=plt.gca())

plt.xlabel('Generation')
plt.ylabel('Cost')
plt.title('Cost components per Generation')
plt.legend()
plt.grid(axis='y') # Add grid lines only on the y-axis for better readability
plt.show()
```



Biomarker Error

```
# Select the relevant error columns for iteration 340
error_df_340 = df_clean[df_clean['iter'] == 705][['Vol_cost', 'ESV_cost', 'EF_error']]

# Generate the LaTeX table
latex_table_340 = error_df_340.to_latex(index=False, float_format="%.4f")

# Print the LaTeX table
print(latex_table_340)
```

```
\begin{tabular}{rrr}
\toprule
Vol_cost & ESV_cost & EF_error \\
\midrule
0.0005 & 0.0073 & 0.0037 \\
\bottomrule
\end{tabular}
```

```
# prompt:   Difference Vol_cost - V0_cost at Gen 8: -0.7003 which iteration gives this in generation_stats

# Find the generation where the difference is approximately -0.7003 at Generation 8
target_diff = -0.7003
target_gen = 8

# Check if the target generation exists in the index
if target_gen in diff_vol_v0.index:
  # Find the absolute difference between the calculated difference and the target difference
  diff_at_target_gen = diff_vol_v0.loc[target_gen]

  if abs(diff_at_target_gen - target_diff) < 1e-4: # Use a small tolerance for floating point comparison
    print(f"\nIteration {target_gen} gives a Difference Vol_cost - V0_cost of {diff_at_target_gen:.4f}")
  else:
    print(f"\nGeneration {target_gen} exists, but the difference is {diff_at_target_gen:.4f}, not {target_d

else:
  print(f"\nGeneration {target_gen} is not in the generation_stats index.")

# If you need to find the generation closest to the target difference at the target generation
# You could iterate around the target generation if needed, but the current data indicates gen 8 is present.
```

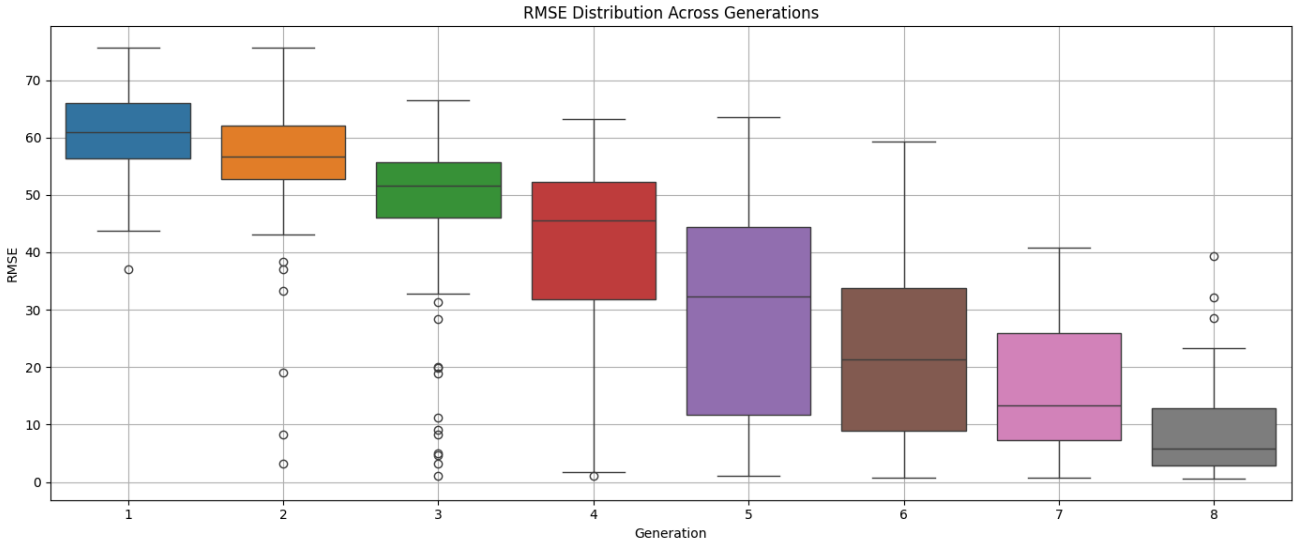
```
# For a more general case, if you wanted to find which generation has a difference closest to -0.7003, you c
# closest_gen = (diff_vol_v0 - target_diff).abs().idxmin()
# print(f"\nThe generation with the difference closest to {target_diff} is Generation {closest_gen}")
# print(f"  Difference at Generation {closest_gen}: {diff_vol_v0.loc[closest_gen]:.4f}")
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-3344549998.py in <cell line: 0>()
      6
      7 # Check if the target generation exists in the index
----> 8 if target_gen in diff_vol_v0.index:
      9   # Find the absolute difference between the calculated difference and the target difference
     10   diff_at_target_gen = diff_vol_v0.loc[target_gen]

NameError: name 'diff_vol_v0' is not defined
```

Boxplot of Volume RMSE across generations

```
plt.figure(figsize=(14, 6))
my_pal = {"0": "red", "1": "green", "2":"blue", "3":"yellow", "4":"pink", "5":"orange", "6":"purple", "7":"l
# Ensure your palette dictionary has keys for all generation numbers present in your data
generations = df_filtered['gen'].unique()
# Create a palette with colors for each generation
my_pal = {str(gen): "C{}".format(i % 10) for i, gen in enumerate(generations)}

sns.boxplot(x='gen', y='Difference_RMSE', data=df_filtered, palette=my_pal)
plt.title('RMSE Distribution Across Generations')
plt.xlabel('Generation')
plt.ylabel('RMSE')
plt.grid(True)
plt.tight_layout()
plt.show()
```

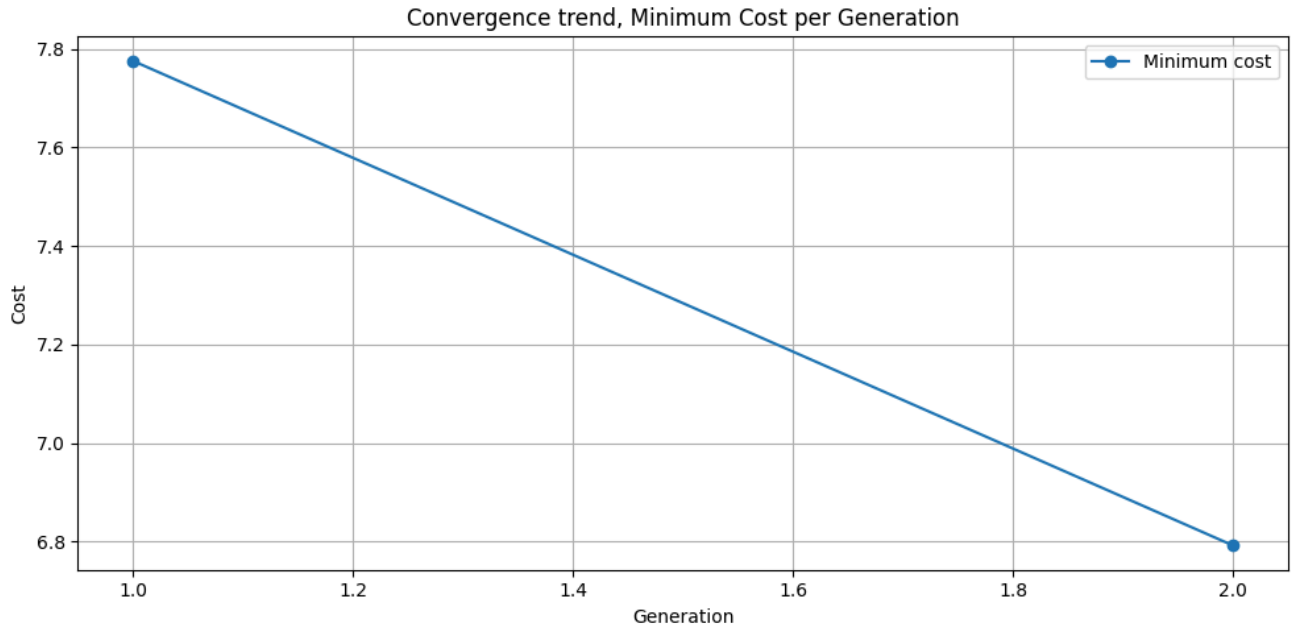
```
/tmp/ipython-input-852465562.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variab

  sns.boxplot(x='gen', y='Difference_RMSE', data=df_filtered, palette=my_pal)
```



PLOT CONVERGENCE TREND

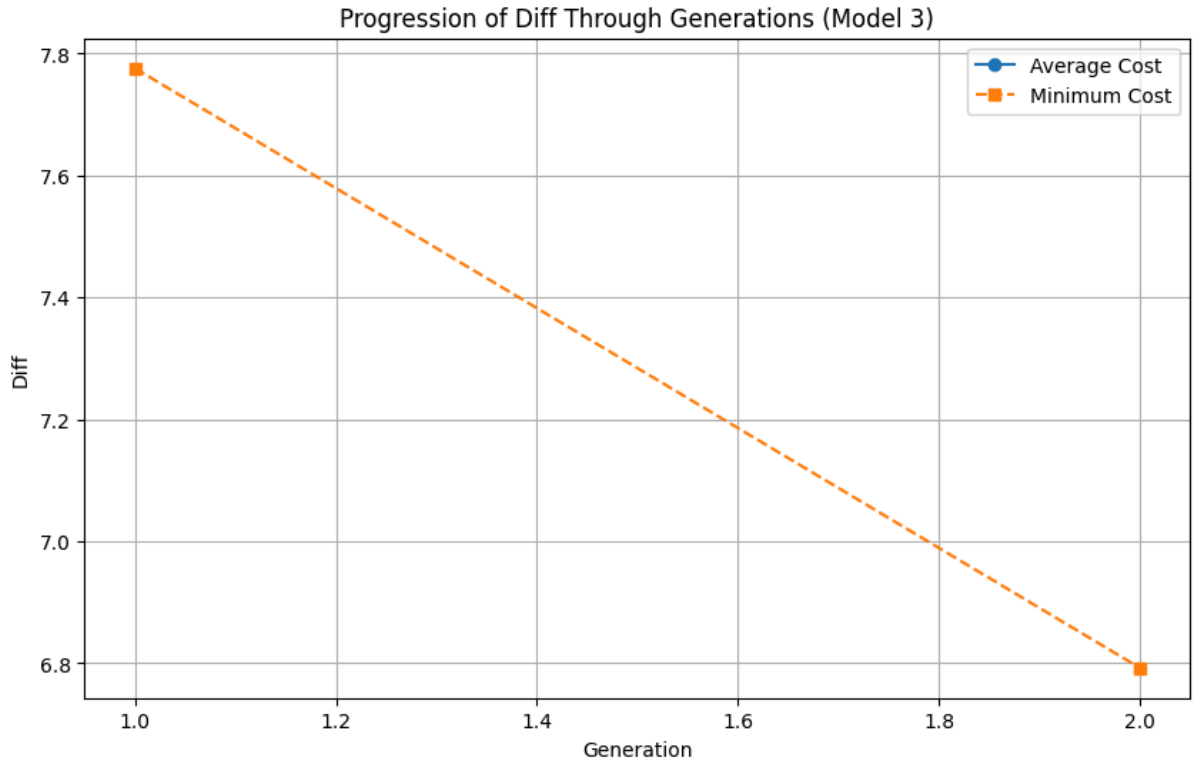
```
min_diff_per_gen = df_clean.loc[df_clean.groupby('gen')['Diff'].idxmin()][['gen', 'Diff']].reset_index(drop=

x_data = min_diff_per_gen['gen']
```

```
y_data = min_diff_per_gen['Diff']

plt.figure(figsize=(10, 5))
plt.plot(x_data, y_data, marker='o', linestyle='-', label='Minimum cost')

plt.title('Convergence trend, Minimum Cost per Generation')
plt.xlabel('Generation')
plt.ylabel('Cost')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



AVERAGE Convergence Trend

```
# Compute average and minimum Diff per generation
avg_diff_gen = df_clean.groupby('gen')['Diff'].mean()
min_diff_gen = df_clean.groupby('gen')['Diff'].min()

# Plot progression of average and minimum Diff through generations
plt.figure(figsize=(10, 6))
plt.plot(avg_diff_gen.index, avg_diff_gen.values, marker='o', label='Average Cost')
plt.plot(min_diff_gen.index, min_diff_gen.values, marker='s', linestyle='--', label='Minimum Cost')
plt.xlabel('Generation')
plt.ylabel('Diff')
plt.title('Progression of Diff Through Generations (Model 3)')
plt.legend()
plt.grid(True)
plt.show()
```



** Exp:control_1_v1_edp_all **

key visual analyses of optimization process:

1. **Minimum Volume RMSE per Generation**:
   - The minimum RMSE decreases early on and plateaus, indicating that the best individuals in each generation converge quickly to optimal or near-optimal volume estimates.
2. **Boxplot of Volume RMSE Across Generations**:
   - The spread (variance) of Volume RMSE narrows with generation, confirming increased consistency in performance.
   - Outliers reduce over time, showing fewer poor solutions as the algorithm evolves.
3. **Vol_cost vs. V0_cost (coloured by Total Difference)**:
   - Most optimal (dark-colored) points lie in the bottom-left, indicating low values of both cost components contribute to minimal total error.
   - There is a visible negative correlation: improving one often means reducing the other, supporting multi-objective behaviour.

Would you like a similar analysis for parameter evolution or strain-related trends next?

Minimum Volume RMSE per generation

For simulation 28, The simulation did not complete Iterations in the 100 Generations. A global minimum was reached at 50% of iteration.

```
from scipy.optimize import curve_fit
from numpy.polynomial import Polynomial

min_rmse_per_gen = df.loc[df.groupby('gen')['RMSE'].idxmin()][['gen', 'RMSE']].reset_index(drop=True)

x_data=min_rmse_per_gen['gen']
y_data=min_rmse_per_gen['RMSE']
```

```
def exp_func(x, a, b, c):
    return a * np.exp(-b * x) + c
#TODO of exp fit required
#params, _ = curve_fit(exp_func, x_data, y_data, p0=(100, 0.1, 5))

#x_fit = np.linspace(x_data.min(), x_data.max(), 200)
#y_fit = exp_func(x_fit, *params)

# Fit a polynomial of degree 3
polynomial = Polynomial.fit(x_data, y_data, 3)

# Generate points for the fitted curve
x_fit_p = np.linspace(x_data.min(), x_data.max(), 200)
y_fit_p = polynomial(x_fit_p)
#y_fit_p = exp_func(x_fit_p, *params) # IF exp fit is required



plt.figure(figsize=(10, 5))
plt.plot(x_data, y_data, marker='o', linestyle='-',label='Results Curve')
#plt.plot(x_fit, y_fit, 'r-', label='Fitted Curve')
#plt.plot(x_fit_p, y_fit_p, 'g-', label='Fitted Curve') # if exp fit is reauires

"""plt.title('Minimum Cost per Generation')
plt.xlabel('Generation')
plt.ylabel('Minimum RMSE')"""

plt.title('Minimum Cost per Generation', fontsize=14)
plt.xlabel('Generation', fontsize=12, fontweight='bold')
plt.ylabel('Minimum Cost', fontsize=12, fontweight='bold')

plt.grid(True)
plt.tight_layout()
plt.legend()
plt.show()
```

```
#Percentage error in volume
# Find the row with the minimum RMSE
min_rmse_row = df.loc[df['RMSE'].idxmin()]

# Calculate the percentage error in volume
target_volume = min_rmse_row['Target Volume']
simulated_volume = min_rmse_row['Simulated Volume']
percentage_error = abs((target_volume - simulated_volume) / target_volume) * 100

print(f"For the minimum RMSE, the percentage error in volume is: {percentage_error:.2f}%")
```

```
#Show the row with Min RSME
print(min_rmse_row)

print("Parameters :")
min_rmse_row.Params

#Iteration 130
# Longitudinal Strain loss
```

GA RMSE per generation

```
min_rmse_per_gen = df.loc[df.groupby('gen')['Difference'].idxmin()][['gen', 'Difference']].reset_index(drop=

x_data=min_rmse_per_gen['gen']
y_data=min_rmse_per_gen['Difference']

def exp_func(x, a, b, c):
    return a * np.exp(-b * x) + c

params, _ = curve_fit(exp_func, x_data, y_data, p0=(100, 0.1, 5))
```

```
x_fit = np.linspace(x_data.min(), x_data.max(), 200)
y_fit = exp_func(x_fit, *params)



plt.figure(figsize=(10, 5))
plt.plot(x_data, y_data, marker='o', linestyle='-')
plt.title('Minimum Difference per Generation')
plt.plot(x_fit, y_fit, 'r-', label='Fitted Curve')

# Add value labels to each point
for x, y in zip(x_data, y_data):
    plt.text(x, y, f"{y:.1f}", ha='center', va='bottom', fontsize=9)

plt.xlabel('Generation')
plt.ylabel('Minimum Difference')
plt.legend()

plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# prompt: show all iterations  iteration with  V0_cost and Vol_cost below 100

# Filter the DataFrame to show only iterations where V0_cost and Vol_cost are below 100
filtered_iterations = df[(df['V0_cost'] < 10) & (df['Vol_cost'] < 10
                                                 )]

# Display the filtered iterations
print("\nIterations with V0_cost and Vol_cost below 100:")
filtered_iterations
```

```
# V0_cost by Vol_cost and a color scale of difference

plt.figure(figsize=(10, 6))
sc = plt.scatter(df["Vol_cost"], df["V0_cost"], c=df["diff"], cmap="viridis", alpha=0.7)
plt.colorbar(sc, label="Total Difference")
plt.xlabel("Vol_cost")
plt.ylabel("V0_cost")
plt.title("Cost Component Interaction (coloured by Total Difference)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# V0_cost by Strain_cost and a color scale of difference

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sc = plt.scatter(df["Strain_cost"], df["V0_cost"], c=df["Difference"], cmap="viridis", alpha=0.7)
plt.colorbar(sc, label="Total Difference")
plt.xlabel("Strain_cost")
plt.ylabel("V0_cost")
plt.title("Cost Component Interaction (coloured by Total Difference)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
# V0_cost by Strain_cost and a color scale of difference

import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
```